

Оценка сложности рефал-функций для разных представлений объектных выражений

Учебное приложение к НИР

Содержание

1	Зачем это нужно	2
2	Кратко о синтаксисе Рефала-5 (достаточно для разбора сложности)	2
2.1	Программа, функция, предложение	2
2.2	Объектное выражение и термы	3
2.3	Переменные образца	3
2.4	Примеры: сопоставление (левая часть) и построение результата (правая часть)	3
3	Операции над выражением как абстрактный тип	6
4	Алгоритм оценки сложности (по шагам)	6
4.1	Шаг 1. Разметить образец	6
4.2	Шаг 2. Подсчитать использование переменных в правой части	7
4.3	Шаг 3. Разобрать правую часть как дерево вызовов	7
4.4	Шаг 4. Рекурсия	7
4.5	Шаг 5. Свести к модели представления	7
4.6	Таблица и «ячейки»: что именно запоминать	7
5	Пять функций-примеров и разбор по шагам алгоритма	8
5.1	Функция 1: длина выражения (Length)	8
5.2	Функция 2: последний символ (Last)	9
5.3	Функция 3: неоднозначность и перебор (Contains)	9
5.4	Функция 4: копирование таблицы (Subst)	10
5.5	Функция 5: рекурсивная конкатенация результата (Map)	10
6	Как завтра отвечать на две присланные функции	11
7	Замечание о точности	11

1 Зачем это нужно

Интерпретатор Рефала на каждом шаге выполняет две группы действий:

1. **Сопоставление** аргумента функции с образцом левой части предложения: разбиение выражения на фрагменты, связывание переменных.
2. **Построение результата** правой части: подстановка значений переменных, вложенные вызовы, конкатенация фрагментов в одно объектное выражение.

Стоимость этих действий в *единицах работы с памятью* зависит от того, **как в памяти представлено объектное выражение**. В записке рассматривались (в упрощённом виде):

- **Классическое (плоское двусвязное)** — узел на каждый знак записи (символ или скобка), быстрые концы и конкатенация, дорогое полное копирование.
- **Компромиссное (подвешенные скобки)** — узел на терм верхнего уровня, копирование верхнего уровня дешевле, но сложнее память.
- **Массивное** — непрерывный блок, *e*-переменная как пара индексов (копирование дёшево), конкатенация часто дорогая.
- **Векторно-списковое** — результат как список фрагментов вектора, конкатенация откладывается.
- **Роле (гипотетически)** — дерево конкатенаций: дешёвая склейка, дороже работа с концами.
- **Раскрученная двусторонняя очередь** — амортизированно дешёвые концы и конкатенация при структурном разделении.

Ниже — **алгоритм мысленной оценки**: не формальное доказательство, а порядок рассуждений, которым можно пользоваться на экзамене или при разборе чужой функции.

2 Кратко о синтаксисе Рефала-5 (достаточно для разбора сложности)

2.1 Программа, функция, предложение

- **Программа** — набор функций.
- **Функция** — имя и блок из одного или нескольких **предложений** в фигурных скобках.
- **Предложение**: образец = результат;
- При вызове интерпретатор просматривает предложения **сверху вниз** и берёт **первое**, образец которого сопоставим с аргументом.

Вызов функции записывают угловыми скобками: <Имя аргумент>.

2.2 Объектное выражение и термы

Объектное выражение — конечная последовательность **термов** (возможно пустая).

Терм бывает:

- **атомарный** (символ в терминологии Рефала): символ-слово (`True`), макродигит (42), литера из строкового литерала (`'a'` — один символ-литера);
- **скобочный**: `( ... )` — внутри снова объектное выражение.

2.3 Переменные образца

- `s.X` — ровно один символ (атомарный терм).
- `t.X` — ровно один терм (в том числе скобочный).
- `e.X` — цепочка термов произвольной длины, в том числе пустая.

Повтор одного и того же имени переменной в образце означает **совпадение значений** (как в листинге с `Contains` ниже).

2.4 Примеры: сопоставление (левая часть) и построение результата (правая часть)

Ниже для **каждого** примера: какой аргумент, как интерпретатор находит первое подходящее предложение (сопоставление), из чего потом собирается результат. Порядок вычисления в правой части: сначала вычисляются вложенные вызовы `<...>` слева направо, затем конкатенация текстовых фрагментов результата в одно выражение.

Вызов и аргумент — не одно и то же.

- **Вызов функции** — целиком запись в угловых скобках `<...>`: имя функции и переданное ей выражение.
- **Аргумент** — только **объектное выражение после имени** (цепочка термов до закрывающей `>`). Имя функции в аргумент **не** входит; угловые скобки тоже не входят — это синтаксис вызова вокруг аргумента.

Иллюстрация на одной строке (пробел после имени — по стилю Рефала-5; его можно опускать, если однозначно):

```
1 <TakeRight A : B C>
```

Здесь **вызов** — вся строка `<TakeRight A : B C>`. **Аргумент** — подвыражение `A : B C` (четыре терма: `A`, `:`, `B`, `C`). Левую часть предложения сопоставляют именно с `A : B C`, а не с именем `TakeRight`.

Пример 1. Пустой аргумент и константа в результате

```
1 EmptyLen {
2   = 0;
3 }
```

Вызов (пример):

```
1 <EmptyLen >
```

(между именем и > нет термов — **аргумент пустой**.)

Аргумент: пустое выражение (ни одного терма).

Сопоставление. В образце первого предложения нет ни символов, ни переменных — это **пустой образец**. Он сопоставим только с пустым аргументом. Переменные не связываются.

Правая часть. Результат — одна макроцифра 0. Дополнительных операций над полем зрения нет.

Пример 2. Один символ с головы и хвост

```
1 HeadEcho {
2   s.H e.T = s.H (s.H) <HeadEcho e.T>;
3   = ;
4 }
```

Вызов (пример):

```
1 <HeadEcho A B>
```

Аргумент первого шага: A B (два символа). На рекурсии — <HeadEcho B>, аргумент B; затем <HeadEcho > с пустым аргументом.

Сопоставление второго предложения. Слева направо: s.H забирает ровно один символ с **начала** — $H \mapsto A$. Затем e.T забирает всё оставшееся до конца аргумента — $T \mapsto B$. (Если бы после s.H был ещё фиксированный терм, граница для e.T сузилась бы.)

Правая часть. Сначала вычисляется <HeadEcho e.T>, т.е. <HeadEcho B>. Потом результат всего предложения — конкатенация трёх фрагментов: s.H, скобочного (s.H) (внутри копия того же символа), и ответа рекурсии. Для B сработает третье предложение = ; — пустой результат. Итог вызова на A B: A (A).

Пример 3. Фиксированные термы-якоря и одна e-переменная

```
1 Inside {
2   'a' e.Mid 'b' = e.Mid;
3 }
```

Вызов (пример):

```
1 <Inside 'a' X Y 'b'>
```

Аргумент: 'a' X Y 'b' (якоря — литеры 'a' и 'b').

Сопоставление. Первый терм аргумента обязан совпасть с 'a' — да. Далее $e.Mid$ должна поглотить середину так, чтобы остаток начинался с 'b'. При единственном вхождении 'b' на конце получается $Mid \mapsto X Y$. Если бы 'b' встречалось дважды, без дополнительных ограничений возможны **несколько** разбиений — перебор.

Правая часть. Результат — одно значение $e.Mid$, т. е. копия/передача подцепочки $X Y$ без якорей.

Пример 4. Повтор имени переменной в образце

```
1 PairEq {
2   s.A s.B s.A = (s.A s.B);
3 }
```

Вызов (пример):

```
1 <PairEq X Y X>
```

Аргумент: $X Y X$.

Сопоставление. $s.A \mapsto X$, $s.B \mapsto Y$, третий символ снова должен быть X — совпадает с уже связанным $s.A$. Иначе предложение не подходит.

Правая часть. Скобочный терм $(s.A s.B)$ — один терм с внутренностью $X Y$.

Пример 5. Две e-переменные и разделитель

```
1 TakeRight {
2   e.Left ':' e.Right = e.Right;
3 }
```

Вызов (пример):

```
1 <TakeRight A : B C>
```

Аргумент: $A : B C$ (не путать с **правой** частью предложения $= e.Right$; — там $e.Right$ уже *имя* куска аргумента после сопоставления).

Сопоставление. Рефал перебирает варианты длины $e.Left$, пока не найдётся такое разбиение, что после выделенной части идёт ':', а за ним — $e.Right$. Здесь единственное разбиение: $Left \mapsto A$, $Right \mapsto B C$.

Правая часть. Возвращается только $e.Right$ — $B C$.

Пример 6. Таблица в скобках и рекурсия (как у Subst)

```
1 CopyTwice {
2   (e.Tab) s.X = (e.Tab) (e.Tab) s.X;
3 }
```

Вызов (пример):

```
1 <CopyTwice (A B) Z>
```

Аргумент: $(A \ B) \ Z$ — два термина верхнего уровня: скобочный $(A \ B)$ и символ Z .

Сопоставление. Первый терм — скобочный; внутри образца $(e.Tab)$ переменная $e.Tab$ связывается с внутренностью таблицы: $Tab \mapsto A \ B. s.X \mapsto Z$.

Правая часть. Конкатенация трёх кусков: две копии $(e.Tab)$ и символ $s.X$. Здесь $e.Tab$ в левой части **один** раз, в правой — **два** — типичный сигнал к вопросу о **физическом копировании** при списковом представлении (см. шаг 2 алгоритма ниже).

3 Операции над выражением как абстрактный тип

Чтобы оценивать сложность, достаточно отслеживать, сколько раз и в каком порядке вызываются типичные операции реализации:

1. **Пусто?** — проверка на пустое выражение.
2. **Отделить начало** — взять первый терм и остаток (для списков часто $O(1)$ на указателях).
3. **Отделить конец** — взять последний терм и остаток (для двусвязного списка $O(1)$, для горки обычно дороже).
4. **Конкатенация** — склеить два выражения в одно (у классического списка $O(1)$, у наивного массива часто $O(|A| + |B|)$).
5. **Сингуляр** — выражение из одного термина.
6. **Копирование значения переменной** — когда одно и то же e (или вся таблица) используется в правой части больше раз, чем в левой: в списковых представлениях часто требуется **физическая копия** ($O(\text{размер})$), в массивном — часто достаточно скопировать **диапазон** как пару индексов ($O(1)$).

4 Алгоритм оценки сложности (по шагам)

Дана функция (или одно предложение). Нужно оценить время **одного успешного применения предложения** и/или всей функции на входе длины n .

4.1 Шаг 1. Разметить образец

Пройти образец слева направо и записать, что происходит с аргументом:

- фиксированный терм в образце — сравнение с текущей позицией (обычно $O(1)$ за терм, если не нужно «отматывать» длинный перебор);
- $s.*$ — отделить один символ с начала (или с конца, если образец устроен так — смотрите на то, как Рефал сопоставляет; на практике важны оба конца);
- $t.*$ — отделить один терм (скобочный терм может потребовать прыжка по паре скобок в плоском представлении — всё ещё часто $O(1)$ на шаг);
- $e.*$ — либо однозначное «съесть всё до известной границы», либо **неоднозначность**: несколько вариантов разбиения.

4.2 Шаг 2. Подсчитать использование переменных в правой части

Для каждой переменной X сравнить:

- k_L — сколько раз X встречается в **левой** части;
- k_R — сколько раз в **правой** части.

Если $k_R > k_L$, то при классическом списковом представлении часто нужно $k_R - k_L$ **копий** большого значения (типичный источник квадратики в Subst).

4.3 Шаг 3. Разобрать правую часть как дерево вызовов

Вложенные вызовы $\langle G \dots \rangle$ сначала вычисляются, их результаты затем конкатенируются. Каждая явная конкатенация в результате — потенциальная линейная работа для массива.

4.4 Шаг 4. Рекурсия

Записать рекуррентное соотношение: на сколько уменьшается размер аргумента за один вызов, сколько конкатенаций/копий на шаг.

4.5 Шаг 5. Свести к модели представления

Подставить стоимость операций из таблицы ниже (порядок, не константы).

Важно. В каждой ячейке таблицы указан порядок **одной** примитивной операции (одно отделение с конца, одна конкатенация двух кусков и т. д.). Оценка функции — это, как правило, «сколько раз эта операция выполняется на всём входе» $\times$ «что в ячейке»: например, $\Theta(n)$ раз по $O(1)$ из столбца «Концы» даёт $O(n)$, а не $O(1)$.

Представление	Концы	Конкатенация	Копия переменной
Классическое списковое	$O(1)$	$O(1)$	$O(N)$
Компромиссное	$O(1)$	$O(1)$	$O(N_{\text{верх}})$
Массивное	$O(1)$	$O(A + B)$	$O(1)$
Векторно-списковое	$O(1)$	отложенная	$O(1)$
Rope	$O(\log n)$	$O(1)$ /баланс	$O(1)$
Раскрученная	$O(1)$ амортиз.	$O(1)$ амортиз.	$O(1)$
двусторонняя очередь			

Таблица 1: Порядки стоимости базовых операций (последняя строка — предлагаемое в НИР представление; схематично)

4.6 Таблица и «ячейки»: что именно запоминать

Таблица — это не одна «клетка на всю функцию», а **справочник стоимости базовых приёмов** для каждого представления:

- **Строка** — как устроено поле зрения (классический список, массив, раскрученная двусторонняя очередь и т. д.).
- **Столбец** — вид операции: работа с **концами** (отделить начало/конец), **конкатенация**, **копия** большого значения переменной.

Как отвечать на вопрос про сложность. (1) По шагам 1–5 (подразделы «Шаг 1»–«Шаг 5» в разделе 4 выше) выписать, какие из этих операций реально возникают при сопоставлении и при построении правой части и сколько раз на один шаг / на весь вход. (2) Для **каждого** интересующего представления подставить порядок из соответствующей *ячейки* таблицы: например, «много конкатенаций подряд + массивное представление» $\Rightarrow$ столбец «Конкатенация» и строка «Массивное». (3) Не забыть ловушки вне таблицы: неоднозначные $e.*$ (перебор), рекурсия с уменьшением размера, $k_R > k_L$ для длинных $e.*$.

Запомнить дословно все числа в таблице не обязательно; важно **смысл**: где конкатенация дешева, где дорога, где копия длинного хвоста дешёва как «срез» индексов.

5 Пять функций-примеров и разбор по шагам алгоритма

Для каждой функции ниже кратко пройдены шаги 1–5 из раздела 4: образец $\rightarrow$ переменные в правой части $\rightarrow$ дерево вызовов $\rightarrow$ рекурсия $\rightarrow$ что взять из таблицы.

5.1 Функция 1: длина выражения (Length)

```

1 Length {
2     = 0;
3     s.First e.Rest = <Add 1 <Length e.Rest>>;
4 }
```

Шаг 1 (образец). Либо пустой аргумент (первое предложение), либо от головы: `s.First` — один символ, `e.Rest` — остаток. Одно отделение начала на успешное применение второго предложения.

Шаг 2 (переменные). `e.Rest` по одному разу в левой и правой части; лишних копий длинного хвоста из-за повторения нет.

Шаг 3 (правая часть). Сначала вычисляется `<Length e.Rest>`, затем `<Add 1 ...>`. Конкатенация поля зрения минимальна (результат — число).

Шаг 4 (рекурсия). Длина аргумента уменьшается на 1 за вызов; всего $\Theta(n)$ вызовов второго предложения для входа из n символов.

Шаг 5 (таблица) — ответ по *каждой строке*. Шагов сопоставления второго предложения $\Theta(n)$; на шаг — одно «отделить начало» (столбец «Концы»), копия длинного `e.Rest` не размножается (шаг 2). Слово «концы» в таблице — про стоимость *одной* примитивной операции, итог = (число шагов) $\times$ (ячейка «Концы»).

- **Классическое списковое.** «Концы» $O(1)$ за шаг $\Rightarrow n \cdot O(1) = O(n)$.
- **Компромиссное.** Для цепочки символов верхнего уровня (без дорогого углубления в скобки) то же поведение, что у списка на верхнем уровне $\Rightarrow O(n)$ при $O(1)$ на шаг.

- **Массивное.** Если `e.Rest` — срез/сдвиг индекса без физического копирования всего хвоста на каждом шаге, «Концы» $O(1) \Rightarrow O(n)$. Если же каждый шаг пересобирает массив под новый хвост, доминируют «Конкатенация»/копия $\Rightarrow$ до $O(n^2)$ (ловушка модели, не ячейка «Концы» одна).
- **Векторно-списковое.** $\Theta(n)$ шагов; при $O(1)$ на сдвиг границы у буфера $\Rightarrow O(n)$ (отложенная конкатенация к результату-числу обычно не добавляет порядка).
- **Роле.** В таблице «Концы» $O(\log n) \Rightarrow n \cdot O(\log n) = O(n \log n)$ для такой рекурсии от головы.
- **Раскрученная двусторонняя очередь** (представление из НИР, не произвольная абстрактная двусторонняя очередь без раскрутки буферов): «Концы» $O(1)$ амортизировано $\Rightarrow O(n)$ амортизировано по числу шагов.

Итог. Для пяти первых строк (без горе) при дешёвом отделении начала обычно $O(n)$; для горе — $O(n \log n)$ по столбцу «Концы»; для массива обязательно оговорить, передаётся ли `e.Rest` срезом или копируется блок.

5.2 Функция 2: последний символ (Last)

```

1 Last {
2   e.Init s.Last = s.Last;
3 }
```

Шаг 1 (образец). `e.Init` поглощает префикс, `s.Last` — последний символ. Часто это заставляет механизм сопоставления «пробовать» длины `e.Init`; в худшем случае — линейное движение по цепочке от головы (зависит от реализации Рефала).

Шаг 2. `s.Last` один раз слева и справа; `e.Init` в результат не входит — копировать длинную цепочку в ответ не нужно.

Шаг 3. Результат — один символ; построение тривиально.

Шаг 4. Рекурсии нет; один проход по предложениям.

Шаг 5 (таблица) — ответ по каждой строке. Одно применение предложения на аргументе из n символов; доминирует не рекурсия, а **стратегия сопоставления** образца `e.Init s.Last` (сколько раз дергается «Концы» при переборе границы `e.Init`).

- **Классическое списковое.** При сопоставлении «от головы» (удлинение `e.Init`, проверка последнего терма) в худшем случае $\Theta(n)$ операций с концами $\Rightarrow O(n)$. Если реализация сразу «смотрит» на хвост (двусвязность) — $O(1)$.
- **Компромиссное.** Аналогично верхнему уровню: сканирование цепочки символов $\Rightarrow O(n)$ в худшем случае; доступ с конца при поддержке со стороны машины — $O(1)$ по верхнему уровню.
- **Массивное.** Доступ к последнему элементу по индексу $O(1)$; если же сопоставление моделируется как многократный сдвиг начала без среза — до $O(n)$. Конкатенация в результате не тянет порядок (один символ).

- **Векторно-списковое.** Обычно $O(1)$ амортизировано к последнему фрагменту + границе буфера или $O(n)$ при наивном проходе — зависит от реализации сопоставления.
- **Роре.** Доступ к «последнему» символу через дерево: ячейка «Концы» $O(\log n)$ на одно успешное сопоставление $\Rightarrow O(\log n)$ за вызов при хорошей реализации; при переборе вариантов границы — умножить на число попыток.
- **Раскрученная двусторонняя очередь (НИР).** Доступ к хвосту и отделение последнего символа амортизировано $O(1)$ при сопоставлении с конца $\Rightarrow O(1)$ амортизировано за одно предложение; сопоставление только с головы — как у списка, $O(n)$ в худшем случае.

Итог. Диапазон от $O(1)$ (хвост, РДО/двусвязность, явная поддержка) до $O(n)$ или $O(\log n)$ за одно предложение — обязательно сказать, *откуда* машина сопоставляет образец.

5.3 Функция 3: неоднозначность и перебор (Contains)

```

1 Contains {
2     s.Needle e.Left s.Needle e.Right = True;
3     s.Needle e.Any                    = False;
4 }
```

Шаг 1 (образец). Первое предложение: `s.Needle`, затем `e.Left`, снова тот же символ `s.Needle`, затем `e.Right`. Неоднозначность: где заканчивается `e.Left` и где второе вхождение `s.Needle` — перебор вариантов (бэктрекинг).

Шаг 2. Все переменные в результате не используются (только `True/False`); проблема копий `e.*` в ответе не стоит.

Шаг 3. Правая часть — константа; дорого только сопоставление.

Шаг 4. Рекурсии нет; число попыток сопоставления первого предложения может быть большим.

Шаг 5 (таблица) — ответ по каждой строке. Пусть в худшем случае $O(n^2)$ кандидатов перебора границы между `e.Left` и вторым `s.Needle`; $T_{\text{канд}}$ — стоимость одного кандидата по столбцу «Концы» (+ при необходимости «Конкатенация» для бэктрекинга). Итог $O(n^2 \cdot T_{\text{канд}})$ до полиномов выше, если $T_{\text{канд}}$ не константа.

- **Классическое списковое.** Сдвиг границы с головы $\Rightarrow T_{\text{канд}} = O(1) \Rightarrow O(n^2)$ в типичной оценке.
- **Компромиссное.** Как у списка на верхнем уровне для цепочки символов $\Rightarrow O(n^2)$ при $O(1)$ на кандидата; если перебор задевает вложенные скобки, константа в $T_{\text{канд}}$ растёт с глубиной структуры.
- **Массивное.** Если каждый кандидат требует сдвига/копирования префикса массива, $T_{\text{канд}}$ может быть $O(n) \Rightarrow$ до $O(n^3)$; при индексах и срезах без копии — ближе к $O(n^2)$.
- **Векторно-списковое.** Часто $T_{\text{канд}} = O(1)$ амортизировано за счёт буферов $\Rightarrow O(n^2)$ по числу кандидатов.

- **Rope.** $T_{\text{канд}} = O(\log n)$ по ячейке «Концы» $\Rightarrow O(n^2 \log n)$ в худшем случае при том же числе кандидатов.
- **Раскрученная двусторонняя очередь (НИР).** $T_{\text{канд}} = O(1)$ амортизированно при сдвиге границы у конца буфера $\Rightarrow O(n^2)$ амортизированно (как у списка, но с учётом редких дорогих ребалансировок внутри длинной серии кандидатов).

Итог. Таблица задаёт $T_{\text{канд}}$; число кандидатов — из образца и входа. Наиболее контрастно: $O(n^2)$ против $O(n^2 \log n)$ (rope) и возможный $O(n^3)$ на наивном массиве.

5.4 Функция 4: копирование таблицы (Subst)

```

1 Subst {
2   (e.Tab)           = ;
3   (e.Tab) s.X e.Rest = <Lookup (e.Tab) s.X> <Subst (e.Tab) e.Rest>;
4 }

```

Шаг 1 (образец). Второе предложение: один скобочный терм с внутренностью $e.Tab$, затем $s.X$, $e.Rest$. Сопоставление с головы: разделить таблицу как один t -образный кусок, затем символ, затем хвост.

Шаг 2. $(e.Tab)$ в левой части **один** раз, в правой — в $\text{<Lookup } (e.Tab) s.X>$ и снова в $\text{<Subst } (e.Tab) e.Rest>$ — **два** раза: $k_R > k_L$ для содержимого таблицы. Это прямое попадание в столбец «Копия переменной» для списковых представлений.

Шаг 3. Сначала **Lookup**, затем рекурсивный **Subst**; между ними конкатенация двух результатов в поле зрения.

Шаг 4. За шаг укорачивается $e.Rest$ на один символ; $\Theta(n)$ шагов при длине аргумента n после таблицы.

Шаг 5 (таблица) — ответ по каждой строке. $\Theta(n)$ рекурсивных шагов по длине $e.Rest$; на шаг — повтор $(e.Tab)$ в правой части (шаг 2) + конкатенация двух кусков + **Lookup**. Пусть $|T| = m$ — размер таблицы (в терминах копируемого объёма для списков).

- **Классическое списковое.** «Копия переменной» $O(m)$ на шаг $\Rightarrow O(nm)$; при $m \sim n$ часто $O(n^2)$. «Конкатенация» мелких к $e.Rest$ — обычно не доминирует.
- **Компромиссное.** Копия по верхнему уровню $O(N_{\text{верх}})$ на шаг $\Rightarrow O(n \cdot N_{\text{верх}})$; вложенность таблицы уменьшает эффективный размер копии по сравнению с классикой.
- **Массивное.** «Копия» таблицы как срез $O(1)$ на шаг; **Lookup** при $O(1) \Rightarrow O(n)$. Конкатенация: если каждый шаг склеивает два массива по ячейке $O(|A| + |B|)$, возможна $O(n^2)$ по длине накапливаемого результата — оговорить модель.
- **Векторно-списковое.** Копия/срез $O(1)$, конкатенации отложенные $\Rightarrow$ часто $O(n)$ + одна материализация; итог зависит от политики **Lookup**.
- **Rope.** Копия указателей на поддерева $O(1)$ за передачу таблицы; конкатенация по таблице $O(1)$; обход для **Lookup** может дать $O(n \log m)$ или $O(n)$ в зависимости от реализации — в оценке оговорить.

- **Раскрученная двусторонняя очередь (НИР).** Как у классического списка по порядку копии подвыражения таблицы, если `e.Tab` материализуется при каждом вложенном вызове — $O(nm)$ в худшем случае; если таблица передаётся структурно без полной копии буферов (модель из записки) — уточнять по инвариантам, ближе к списку или к массиву по копии.

Итог. Для `Subst` решает столбец «Копия переменной» $\times n$ шагов; массив и векторно-списковое снимают копию таблицы; горе добавляет логарифмические факторы к до-ступу; РДО — как у «списковых» сценариев, если не ввести явно дешёвую передачу таблицы без полной копии.

5.5 Функция 5: рекурсивная конкатенация результата (Map)

```

1 Map {
2   = ;
3   s.X e.Rest = <F s.X> <Map e.Rest>;
4 }

```

Шаг 1 (образец). Как у `Length: s.X, e.Rest` — одно отделение начала.

Шаг 2. `e.Rest` не размножается в правой части лишний раз; опасность не в копии `e.*`, а в повторяющейся **конкатенации** результатов.

Шаг 3. Правая часть — конкатенация результата `<F s.X>` с результатом `<Map e.Rest>`. Порядок: сначала внутренние вызовы, потом склейка.

Шаг 4. n рекурсивных уровней при n символах.

Шаг 5 (таблица) — ответ по *каждой строке*. n уровней рекурсии; на уровень — одно отделение начала (как у `Length`) + одна конкатенация результата `<F s.X>` с уже накопленным хвостом. Пусть `<F s.X>` за $O(1)$ по размеру поля зрения.

- **Классическое списковое.** «Конкатенация» $O(1) \Rightarrow O(n)$ суммарно; «Концы» для `s.X/e.Rest` тоже $O(1)$ на шаг.
- **Компромиссное.** Как у списка на верхнем уровне $\Rightarrow O(n)$ при $O(1)$ на `F` и конкатенацию верхнего уровня.
- **Массивное.** «Конкатенация» по ячейке $O(|A|+|B|)$: длина хвоста растёт $1, 2, \dots, n \Rightarrow \sum_{i=1}^n i = O(n^2)$ при наивной материализации после каждого шага.
- **Векторно-списковое.** Конкатенация отложенная $\Rightarrow O(n)$ за накопление списка фрагментов + $O(n)$ или $O(n \log n)$ за финальную материализацию — зависит от политики, но квадратику снимает.
- **Роре.** «Конкатенация» $O(1)$ за склейку; «Концы» при необходимости смотреть в глубину дерева могут дать $O(\log n)$ на шаг $\Rightarrow$ до $O(n \log n)$ вместе с отделением, если доминирует горе.
- **Раскрученная двусторонняя очередь (НИР).** «Концы» и «Конкатенация» по таблице амортизированно $O(1) \Rightarrow O(n)$ амортизированно при $O(1)$ на `F`.

Итог. Доминирует столбец «Конкатенация» $\times n$ уровней (кроме случаев отложенной склейки); массив — главный контраст ($O(n^2)$); горе — возможен дополнительный $\log n$; РДО и классический список — линейно при дешёвой склейке.

6 Как завтра отвечать на две присланные функции

1. Кратко описать, что делает каждое предложение (1–2 предложения на русском).
2. Для **каждого** представления из записки: перечислить, какие из шести операций (раздел 3) возникают и сколько раз на один вызов / на весь вход; затем для каждой такой операции смотреть соответствующую **ячейку** таблицы (строка — представление, столбец — вид работы).
3. Отдельно отметить «ловушки»: $k_R > k_L$ для $e.*$; неоднозначные $e.*$; цепочка конкатенаций в правой части.
4. Дать итог в виде $O(\cdot)$ с явными n, m (размер таблицы, глубина скобок и т.д.).

7 Замечание о точности

Такая оценка отражает **стоимость работы с представлением выражения**. Полное время включает также поиск предложения, встроенные функции, сборку мусора на JVM и т.д. На защите обычно достаточно честно обозначить: «оцениваю доминирующие операции над полем зрения при такой-то модели представления».